Orbitor PRD v1.1 — Connectivity, Growth & Orbitor Plus

Oct 13, 2026 · @Anmol

Decisions

Orbitor launches **Android-first**, keeps **all offline features free**, and sells a **₹99/month Orbitor Plus** whose value is the internet layer that offline-only users cannot get. React Native is not the blocker: Expo Modules let us write Swift and Kotlin for every radio API we need.

1. **Android-first launch (iOS in v1.5).** Android is the large majority of phones in India, and Android↔Android has a mature fast path (Google Nearby Connections). iOS builds are used for our own testing from day one (see Distribution), but are not released to the public yet.
2. **Free core, paid reach.** Offline chat, PTT voice and file fling must stay free; they are the hook and the invite loop. Plus charges for what offline cannot do: reaching friends who are not nearby, backup, and large or long-lived transfers.
3. **Network effect before revenue.** An offline app is useless alone. Growth targets are built around dense clusters (campus, hostel, trek group, event), not scattered installs.
4. **Honest numbers.** 1,000 paying users needs roughly 25,000–40,000 monthly actives at a normal 2.5–4% conversion, not 10,000. See Revenue math.

What changed in this version: the app ships as a signed APK from our own website, not the Play Store. Payments run on the website and the app unlocks Plus when the user logs in. The code is structured so iOS, Play Store billing and new Kotlin or Swift modules can be added later without rewrites.

React Native reality

React Native only limits us where we rely on ready-made JS libraries; anything the OS exposes, we can reach with our own Swift or Kotlin module behind a TurboModule. The real limits are the OS radio rules, which hit native apps equally.

Capability	Reachable from RN?	How	Real limit (not RN's fault)
BLE scan + advertise + GATT server	Yes, custom module	Kotlin BluetoothLeAdvertiser / Swift CoreBluetooth	Off-the-shelf libs are central-only; iOS throttles background advertising
Android↔Android fast link	Yes	Wrap Google Nearby Connections in an Expo module	Needs Google Play Services; Android only
iOS↔iOS fast link	Yes	Wrap Multipeer Connectivity	Apple only
Wi-Fi Aware	Yes, custom module	Android 8+ and iOS 26+ native APIs	Hardware and OEM dependent; test early
Hotspot join (cross-platform)	Yes	Android LocalOnlyHotspot, iOS NEHotspotConfiguration	iOS shows a system prompt; slow setup
LAN (same router)	Yes	mDNS/UDP via a native module	Router client isolation blocks many hostel and public Wi-Fi
Background work	Partly	Android foreground service; iOS background BLE modes	iOS cannot run a sustained scan in background
Audio (Opus, low latency)	Yes, custom module	Oboe + AVAudioEngine	JS must never touch audio frames

Rule: if a library covers only half of a radio API (like central-only BLE), write the module ourselves; this is the planned differentiator in the original PRD, not a workaround.

Transport plan by device pair

The TransportManager picks a link per pair and always keeps BLE as the control channel. Ship the Android↔Android row first; each other row is a later release gated by the Milestone 2 spike.

Pair	Discovery	Chat	Files and voice	Release
Android ↔ Android	BLE + Nearby Connections	Nearby payload	Nearby (Wi-Fi Direct/Bluetooth upgrade)	v1.0
Any ↔ Any, same Wi-Fi	mDNS/UDP	TCP socket	TCP socket	v1.0
Android ↔ Android, no Play Services	BLE	BLE GATT / L2CAP	Hotspot join	v1.1
iOS ↔ iOS	BLE + Multipeer	Multipeer	Multipeer	v1.5
iOS ↔ Android	BLE	BLE GATT / L2CAP	Android hosts LocalOnlyHotspot, iOS joins; Wi-Fi Aware where both support it	v1.5

Offline Mode UX: the user-selected modes from the Network Modes doc stay (Offline, LAN, Internet), but add an **Auto-upgrade toggle inside Offline Mode** that only upgrades between radio links, never to the internet. This keeps the "zero internet traffic" promise.

Offline app sharing: include "Share Orbitor" that sends the APK over Nearby/Bluetooth to a friend with no data. In India this removes the biggest install barrier for the exact moment the app is needed. Because we distribute from our own site, this is allowed; the receiver must enable "install unknown apps" once.

Making it better for users

People install an offline app for a moment (a trek, a fest, an outage) and keep it only if it also works on a normal day. These additions target both.

Feature	Why it drives users	Priority	Tier
Event Rooms: join a room by QR at a fest, campus or trek; everyone in range sees a shared channel	Creates density on day one, no friends needed	P0	Free
Campus mode: LAN chat and file drop on hostel Wi-Fi with no internet	Daily use, not just emergencies	P0	Free
Share Orbitor offline (APK over Nearby)	Removes the install barrier at the moment of need	P0	Free

Feature	Why it drives users	Priority	Tier
SOS and local alerts to everyone in range	Safety story; press and word of mouth	P1	Free
Invite loop: QR pairing shows "Add to Orbitor" for anyone nearby	Every chat creates a new user	P0	Free
Delivery to out-of-range friends over internet when online	The pain of "they are not near me"	P0	Plus
Encrypted backup and new-phone restore	Trust and retention	P1	Plus
Cloud Drop: send large files to a link that lasts 7 days	Replaces WhatsApp's file limits	P1	Plus
Larger groups (up to 50) and extra Event Rooms	Clubs, trek groups	P1	Plus

Daily-use hook: the app must open on a normal day. Campus mode, Event Rooms and Plus delivery make Orbitor useful when the network is fine, so retention does not depend on disasters.

Growth plan: 50k downloads, 10k monthly actives

Target a 20% monthly-active rate on downloads (50k × 20% = 10k). That is only realistic if installs arrive in clusters, so every channel below recruits groups, not individuals.

Channel	Tactic	Downloads goal	Why it clusters
Colleges	30 campus ambassadors; hostel and fest "Orbitor Rooms"; exam-week LAN file drop	20,000	One hostel of 200 students is a ready network
Fests and events	Free event rooms for organisers; QR posters at entry	10,000	Thousands in one place with bad signal
Trek and travel groups	Partner with trek organisers and Instagram trek communities; offline-maps-style content	5,000	Groups of 6–20 who need it together

Channel	Tactic	Downloads goal	Why it clusters
Invite loop	Every pairing and Share Orbitor offers a free invite	10,000	Each active user brings 0.5–1 more
Content and press	Demo video (3 phones, airplane mode), Hindi and English reels, a launch post on Product Hunt and Indian tech communities	5,000	Proof of the 60-second demo

Activation metric: a user is activated when they complete one real exchange with another person (not just install). Track installs → opens → first peer seen → first message → day-7 return.

Funnel targets: install to first peer ≥ 60%; first peer to first message ≥ 70%; day-30 retention ≥ 25% for users in a cluster of 5 or more.

Beta first: run a 500-user closed beta on 2 campuses before the public launch, to fix OEM Bluetooth issues and prove the cluster retention number.

Orbitor Plus: ₹99/month

Plus sells **reach and safety for the moments you are not near your friends**. The free app always works offline; Plus never gates a radio feature, so nobody pays to be stranded.

Plus benefit	What the user gets	Cost to us	Notes
Orbitor Link	Messages reach paired friends over the internet when they are out of range, with end-to-end encryption	Low (small WebSocket relay, ciphertext only)	The core reason to pay
Encrypted backup	Chat and key backup, restore on a new phone	Low	Retention and trust
Cloud Drop	Files up to 2 GB to a 7-day link, no recipient app needed	Medium (storage and egress)	Cap per month to protect margin
Bigger rooms	Groups up to 50, 5 simultaneous Event Rooms	Low	Clubs and trek leads

Plus benefit	What the user gets	Cost to us	Notes
Priority SOS	Alert also pushes over internet to saved contacts	Low	Safety value that sells itself
Perks	Radar themes, custom avatar effects, supporter badge	None	Emotional value, no cost

Plans: Free · Plus ₹99/month · optional Plus annual ₹799/year (a 33% discount, improves cash flow and retention).

Event pack (B2B, later): organisers pay a flat fee for a branded Event Room, a stage broadcast channel and attendee analytics (counts only, no message content). One fest can cover as much revenue as 100+ Plus users.

Pay only when it works

The subscription is only fair if the user can see it work before and while they pay. These rules make that a product requirement, not a promise.

1. **Show before you charge.** Free tier includes 10 Orbitor Link messages per month and one 100 MB Cloud Drop, so users feel the value first.
2. **7-day free trial** of full Plus, no charge until day 8, with a reminder notification on day 6.
3. **Prompt at the moment of need.** When a message to an out-of-range friend is queued, show "Deliver over internet with Plus", not a generic paywall on launch.
4. **Works-or-refund.** If Link delivery or backup fails repeatedly in a month (tracked by our own success metrics), the next month is waived automatically.
5. **Cancel in two taps** inside the app, linked to the store's subscription page; no dark patterns.
6. **Billing by platform.** Users pay on our website through a gateway such as Razorpay (UPI AutoPay and cards), then log in inside the app to unlock Plus. No store takes a cut. If the app later goes on the Play Store, digital goods there must use Play Billing, so keep billing behind the entitlement service (see Code architecture).

Plus status screen: show "Delivered over internet: 42 messages this month" so the user sees the value they paid for.

Revenue math

Selling on our website removes the 15% store fee, so one ₹99 subscriber nets about ₹81.56 if we are GST-registered: ₹99 includes 18% GST (₹83.90 left) and the gateway takes about 2.36% (2% plus GST on the fee, ₹2.34). If we are below the GST registration threshold and unregistered, we keep about ₹96.66. All figures are approximate; confirm GST, gateway pricing and income tax with an accountant. The tables use the conservative ₹81.56.

Monthly actives	Conversion to Plus	Subscribers	Net monthly revenue (approx.)
10,000	2%	200	₹16,300
10,000	3%	300	₹24,500
25,000	3%	750	₹61,200
40,000	2.5%	1,000	₹81,600

Reading it: website checkout converts lower than one-tap store billing, so plan on 2–3% of monthly actives, not 5%. 1,000 subscribers needs about 40,000 monthly actives at 2.5%. Costs and profit are in the next section.

Install friction: sideloading from a website loses users at the "install unknown apps" and Play Protect steps. Assume 30–50% of website visitors never finish installing, so 50,000 completed installs needs roughly 75,000–100,000 download-page visits. Share Orbitor offline (APK over Nearby) bypasses the website and is the cheapest install channel.

Levers if conversion lags: annual plan at ₹799, campus group plans (₹49 per head for 20+ students), the Event pack, and a later Play Store listing for discovery with Play Billing behind the same entitlement service.

Fixes, milestones and metrics

Fixes to the Network Modes doc

- `expo-nearby-connections` as the only Offline transport will not cover iOS↔Android. Use it for Android↔Android only, behind the TransportManager from PRD §6.
- The LAN snippet ignores that many routers isolate clients; add a fallback message and a manual "join hotspot" path.
- The WAN design discovers peers by GPS radius (PostGIS). That conflicts with the PRD non-goal of public stranger discovery and creates stalking risk. In v1, Internet mode reaches **paired friends and rooms only**; any radius discovery needs consent, coarse locations and rate limits first.

- Add a mode-aware outbox: a message queued in Offline mode must never silently go over the internet.

Milestone changes (8 weeks → 12 weeks to public launch)

Week	Deliverable
1–2	Original milestones 1–2, with the Android↔Android Nearby spike first
3–5	BLE discovery, QR pairing, Event Rooms, chat with outbox, Android file fling
6–7	PTT voice, Campus (LAN) mode, Share Orbitor offline
8–9	Account login, entitlement tokens, website checkout, Orbitor Link relay behind a canary flag, backup
10	Closed beta on 2 campuses (500 users)
11–12	Fix beta issues, website download page, signed APK and update manifest, launch
After	iOS and cross-platform hotspot path (v1.5)

New success metrics

Metric	Target
Downloads in first 6 months	50,000
Monthly actives	10,000
Day-30 retention in clusters of 5+	≥ 25%
Free-to-trial conversion	≥ 8% of monthly actives
Trial-to-paid	≥ 40%
Plus subscribers	300 at month 6, 1,000 at month 12

Distribution and iOS testing

Android ships as a signed APK from our website. iPhones are used for testing from week 2 so the code never becomes Android-only, but the public cannot install iOS apps from a website in India, so public iOS waits for the App Store (v1.5).

Channel	Platform	Use	Limit	Cost
Website APK	Android	Public release	User allows unknown apps; Play Protect may warn	₹0
adb / Android Studio builds	Android	Daily development	Our own devices	₹0
Xcode build to own iPhone, free Apple ID	iOS	Daily development	Build expires after 7 days	₹0
TestFlight, paid developer account	iOS	Beta with friends	Up to 10,000 testers; builds last 90 days	$99/year (about ₹750/month)
Ad hoc build	iOS	Internal	100 registered devices per year	Included
App Store	iOS	Public, later	Review; in-app purchase rules for digital goods	Included

Website download page: shows the APK version, its SHA-256 checksum, step-by-step install screenshots, and a "Share offline" button. The app checks a signed update manifest (version, checksum, rollout percent) and downloads updates itself.

Developer verification: Google has announced that apps installed on certified Android devices will need a verified developer identity, starting in a few countries in 2026 and wider after. Check the current status for India and register early so the sideload path does not break.

Code architecture for iOS, paid features and new native modules

One rule drives the structure: JavaScript talks only to typed interfaces, and each platform fills them in natively. That lets the same app run on an iPhone for testing, lets paid features switch on without a rewrite, and lets us add a Kotlin or Swift API later by adding one method.

```
orbitor/
  apps/mobile/             Expo app (Android release, iOS test builds)
  packages/
    core/                  Protocol frames, crypto orchestration, chat and
outbox logic (no RN imports)
    transport/             ITransport contracts, TransportManager, capability
negotiation, MockTransport
    entitlements/          Plans, features, flags, signed-token verification
    ui/                    Skia radar, gestures, design system
  modules/
    orbitor-ble/           Kotlin + Swift (Expo module, TurboModule spec)
    orbitor-transport/     Kotlin: Nearby, Wi-Fi Aware, hotspot. Swift:
Multipeer (stub at first)
    orbitor-audio/         Oboe (Android), AVAudioEngine (iOS)
  services/api/            Fastify: auth, entitlements, relay, backup
  web/                     Marketing site, checkout, download page
```

Rule	What it means in code
Typed native contracts	Every native method has a TypeScript spec. Adding an API RN lacks = add it to the spec, write Kotlin and Swift, no JS call sites change.
Capabilities, not platform checks	`capabilities()` reports what the device supports. iOS returns "unsupported" for radios not built yet, so chat, UI and storage still run on an iPhone.
Mock transport	`MockTransport` simulates peers in simulators and CI; the same test suite runs against it and against real modules.
Offline MVP needs no account	Identity is a local key. Login only appears when the user opens a Plus feature, and then links the account to that key.
Entitlements, not plan checks	The server signs a token (plan, features, expiry). The app embeds the public key and verifies it offline, with a 7-day grace period. Code calls `can('cloud_drop')`, never `plan === 'plus'`.
Billing behind entitlements	Website checkout (Razorpay webhook) creates the entitlement. Play Billing or App Store purchases later create the same record.

Rule	What it means in code
Flags with safe defaults	Signed remote-config JSON is cached on the device and has built-in defaults, so the app starts fully offline. Rollout is by hash of the device ID.
Internet code ships dark	The WAN transport and Plus features are compiled in but off by default behind the `internet_link` flag, so the MVP stays offline-first.

Production deployment and canary releases

Two things get canaried separately: the **backend** (relay, auth, entitlements) and the **app** (APK plus feature flags). The internet feature is released by flag, so turning it on for 5% of users needs no new APK.

Pipeline (GitHub Actions, free tier)

1. Push to main: lint, type-check, unit tests, MockTransport contract tests.
2. Build a signed release APK (keystore in encrypted secrets) and Docker images for the API.
3. Upload the APK and its manifest to Cloudflare R2; push images to GitHub Container Registry.
4. Deploy to staging (second Docker Compose stack on the same VPS), then run smoke tests.
5. Promote to production through the canary stages below.

Backend canary: Caddy (or Nginx) in front of two containers, stable and canary, with weighted routing and a sticky user ID so one user does not flip between versions. Database migrations follow expand-then-contract so both versions work on the same schema. Postgres is backed up daily to R2.

Stage	Audience	Bake time	Promote when	Roll back if
Internal	Team devices, nightly APK	Continuous	CI green	Any test fails
Closed beta	100–500 invited users, beta APK link	2 weeks	Crash-free sessions ≥ 99%	Crash-free < 98%
Canary	5% of devices by manifest rollout percent	48 hours	Crash-free ≥ 99.5%, API errors < 1%	Crash-free < 99% or 5xx > 2%

Stage	Audience	Bake time	Promote when	Roll back if
Staged	25%, then 50%	48 hours each	Same as canary	Same as canary
Full	100%	n/a	n/a	n/a

Internet feature rollout (flag `internet_link`): allowlist of beta Plus users → 1% → 10% → 50% → 100%, with a kill switch that disables it for everyone within one config refresh.

Rollback reality: Android does not install a lower version code over a higher one. Rolling back an APK means turning off flags and shipping a hotfix with a higher version code. Backend rollback is instant by moving traffic back to the stable container.

Monitoring: Sentry for crashes (free tier at first), UptimeRobot for the API and website, and a simple dashboard of delivery success rate, relay latency and active Plus users. The refund rule from earlier reads from the same delivery metrics.

Running cost and profit

Orbitor covers its running costs at about 60 paying subscribers and earns roughly ₹69,000 a month profit at 1,000 subscribers, before income tax. Costs below are estimates in rupees per month, using ₹90 per US dollar.

Monthly cost	Beta (500 users, 0 paid)	100 subscribers	300 subscribers	1,000 subscribers
VPS (API, relay, Postgres, canary container)	1,000	1,500	2,000	3,500
Cloudflare R2 storage (APK, Cloud Drop; no egress fees)	0	150	400	1,400
Domain and transactional email	200	300	300	600
Apple developer account ($99/year)	750	750	750	750
Sentry (free tier until scale)	0	0	0	2,300
Accountant and GST filing	0	1,500	1,500	2,000

Monthly cost	Beta (500 users, 0 paid)	100 subscribers	300 subscribers	1,000 subscribers
Contingency (15%)	290	630	740	1,580
Total cost	**2,240**	**4,830**	**5,690**	**12,130**
Net revenue (₹81.56 per subscriber)	0	8,160	24,470	81,560
Profit before income tax	**−2,240**	**3,330**	**18,780**	**69,430**
Margin	n/a	41%	77%	85%

Break-even: about 60 subscribers cover the 100-subscriber cost level (₹4,830 ÷ ₹81.56), and about 28 cover the beta cost level.

One-time costs: privacy policy and terms (₹5,000–15,000), GST or business registration help (₹2,000–10,000), and a budget Android test phone if you do not have one. Budget ₹10,000–25,000 in total.

Not included: your own salary, income tax, marketing spend (campus ambassadors and posters), refunds under the works-or-refund rule, and customer support time. Subscribers who cancel (assume 8–10% churn a month) mean you need about 100 new trials a month just to hold 1,000.

What moves profit most: conversion rate and churn, not server cost. Costs fall from 59% of revenue at 100 subscribers to 23% at 300 and 15% at 1,000, so the work that matters is retention and the Plus delivery success rate.

Risks and open questions

Risk	Impact	Mitigation
Offline app has no value for the first user in a cluster	High	Event Rooms, campus mode and Plus delivery give solo value on normal days
Subscribers expect offline features to be paid	Medium	Keep every radio feature free and say so on the paywall
Plus conversion below 2%	High	Annual plan, campus group plans, Event pack revenue

Risk	Impact	Mitigation
Sideload friction: Play Protect warnings, unknown-app install steps and Google's developer verification rules	Medium	Just-in-time permission copy; check the declaration forms in week 8
OEM Bluetooth bugs on budget Android phones	Medium	Device matrix and the 500-user beta before launch
Relay abuse and spam over the internet layer	Medium	Friends and rooms only, signed messages, rate limits
GST and payments compliance for a ₹99 digital subscription	Medium	Confirm with an accountant before launch

Open questions

- Is Android-first acceptable for your first users, or do you need iPhones on day one for demos?
- Which two campuses or events can host the beta?
- Do you want the B2B Event pack in v1 or after launch?